ARCHITECTURE REVIEW • V1.0

PROJECT AETHELGARD

Sovereign Cognitive Twin & Cryptographic Provenance Engine

DOC-ID: AETH-HLD-001
TARGET: POSIX / WSL2 / LOCAL
HORIZON: 10-YEAR DURABILITY

CORE STRATEGIC THESIS

Foundation models are rapidly depreciating commodities; **structured personal context and cryptographically verified execution history** are compounding sovereign assets. By capturing decisions, architectural rationales, and execution logs in a model-agnostic local format today, any frontier model in 2036 immediately inherits a decade of your proprietary institutional memory.

1. Executive Summary & Vision

SECTION 01

Modern AI workflows suffer from severe platform lock-in and ephemeral context. Developers and researchers scatter high-value reasoning across stateless cloud chat wrappers, proprietary vector databases, and siloed IDE sessions. When APIs change pricing, alter privacy policies, or deprecate models, years of accumulated problem-solving context vanish.

Project Aethelgard solves this by decoupling *cognitive state* from *model inference*. It operates as an offline-first daemon that passively indexes your digital exhaust—Git commits, architectural decision records (ADRs), research notes, and terminal executions—into an immutable, hash-chained local database, exposing curated memory slices to any AI agent via the Model Context Protocol (MCP).

2. The Four Architectural Invariants

SECTION 02

Event-Sourced Truth

INV-01

Embedding models and graph ontologies become obsolete every 18 months. Raw events are stored in an append-only SQLite ledger. Vector indexes and knowledge graphs are treated as disposable materialized views that can be wiped and rebuilt anytime.

Zero-Cloud Sovereignty

INV-02

No Docker containers, background JVM servers, or external API dependencies for storage. All state lives in `~/.local/share/aethelgard/` using embedded engines (`SQLite` , `sqlite-vec` , `KùzuDB`).

Cryptographic Provenance

INV-03

Every ingested event and agent action is linked in a `BLAKE3` Merkle chain (H_n = BLAKE3(H_{n-1} || $Payload_n$)) and optionally signed with a local Ed25519 key for non-repudiation.

Protocol-First Interop

INV-04

Inference engines are treated as swappable CPU cores. Local 3B–8B models handle background indexing, while frontier models access memory strictly via standardized MCP tools over `stdio` or Unix sockets.

3. End-to-End Topology & Data Flow

SECTION 03

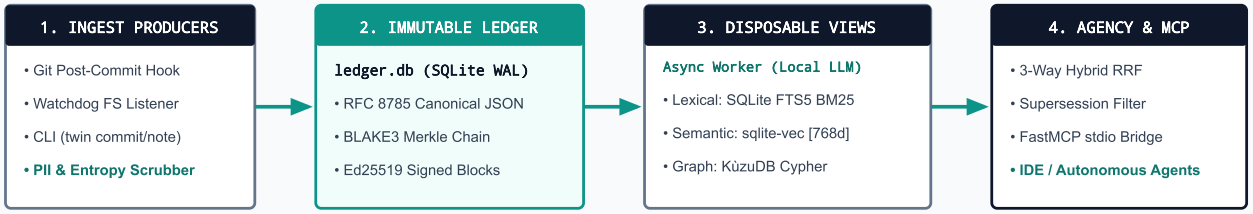


FIG 1.0 — DECOUPLED INGESTION, MERKLE STORAGE, AND MATERIALIZATION PIPELINE

LATENCY: <5ms WRITE / ASYNC INDEX

4. Storage Architecture & Technology Stack

SECTION 04

Every component in the Aethelgard stack is selected for embedded execution, POSIX portability, and open file formats. If a component is superseded in five years, the underlying data remains readable via standard SQLite tools and raw JSONL streams without container orchestration.

SUBSYSTEM LAYER	EMBEDDED TECHNOLOGY	ARCHITECTURAL ROLE & 10-YEAR DURABILITY RATIONALE
1. Privacy Airlock	`Regex + Shannon Entropy`	Intercepts raw payloads pre-ingestion, redacting API keys (`sk-...` , `AKIA...`) and secrets before hashing into the immutable log.
2. Event Ledger	`SQLite 3 (WAL Mode)`	Library of Congress Recommended Format. Stores canonicalized JSON events (RFC 8785) with zero external server overhead.
3. Provenance Chain	`BLAKE3 + Ed25519`	Sub-millisecond cryptographic hashing linking each row to `prev_hash` , providing tamper-evident verification across decades.
4. Lexical Index	`SQLite FTS5 (BM25)`	Captures exact identifier tokens, function signatures, error codes, and financial tickers that dense embeddings miss.
5. Semantic Index	`sqlite-vec (vec0)`	Pure-C, zero-dependency vector search extension running inside SQLite. Eliminates external vector database clusters.
6. Episodic Graph	`KùzuDB (Embedded)`	The "SQLite for Graph Databases." Executes Cypher traversals over temporal edges (`SUPERSEDES` , `CAUSED` , `INVOLVES`) from disk.

5. Hybrid Retrieval & Temporal Supersession

SECTION 05

Standard RAG pipelines fail over multi-year timelines because they treat a deprecated decision from three years ago as equally valid to an architectural change made yesterday. Aethelgard resolves this via **3-Way Reciprocal Rank Fusion (RRF)** combined with graph-based supersession penalties:

3-WAY HYBRID RRF SCORING FUNCTION

$$\text{Score}(d) = \sum_{m \in \{\text{BM25, Vec, Graph}\}} [\, w_m \,/\, (k + \text{rank}_m(d)) \,] \times \Psi_{\text{supersede}}(d) \times e^{-\lambda \Delta t}$$

```
k = 60 (RRF constant)
Ψ = 0.15 if superseded else 1.0
```

6. Model Context Protocol (MCP) Tool Contract

SECTION 06

MCP TOOL NAME	SIGNATURE PARAMETERS	AGENTIC EXECUTION BEHAVIOR
`recall_context`	`query, domain?, include_superseded?`	Executes 3-Way Hybrid RRF across FTS5, `sqlite-vec`, and KùzuDB; filters out obsolete patterns by default.
`commit_decision`	`title, context, rejected[], decision, supersedes_id?`	Hashes an Architectural Decision Record (ADR) into the ledger and writes `SUPERSEDES` edges into KùzuDB.
`trace_entity_history`	`entity_name, max_depth?`	Traverses KùzuDB to return a chronological timeline of commits, bugs, and strategy shifts for a module or concept.
`log_execution_artifact`	`action_type, inputs, outputs, status`	Appends autonomous agent executions (deployments, backtests, migrations) into the Merkle chain for auditability.
`verify_provenance`	`start_seq?, end_seq?`	Recomputes `BLAKE3` hashes and Ed25519 signatures from Genesis to detect database tampering or corruption.

7. Phased Implementation & Risk Mitigation

SECTION 07

- **Phase 1 — Cryptographic Core & Airlock (Weekend 1):** Initialize the Python `uv` workspace, build the regex/entropy secret scrubber, implement RFC 8785 JSON canonicalization, and verify the SQLite WAL `BLAKE3` Merkle chain with tamper-detection unit tests.
- **Phase 2 — Materialized Views & Local Worker (Weekend 2):** Integrate `sqlite-vec` and embedded `KùzuDB`. Connect the low-priority async worker to local Ollama inference (`nomic-embed-text` + `qwen2.5-coder:7b`) and build the `twin rebuild-views` command.
- **Phase 3 — Hybrid RRF & FastMCP Bridge (Weekend 3):** Implement the 3-way RRF ranker with `SUPERSEDES` graph decay, expose the 5 MCP tools over `stdio`, and mount the server inside your primary IDE and CLI workflows.
- **Phase 4 — Passive Daemon & Git Hooks (Weekend 4):** Deploy non-blocking global Git `post-commit` hooks (filtering lockfiles and build artifacts) and register the Linux/WSL2 `systemd` user service for zero-friction background capture.